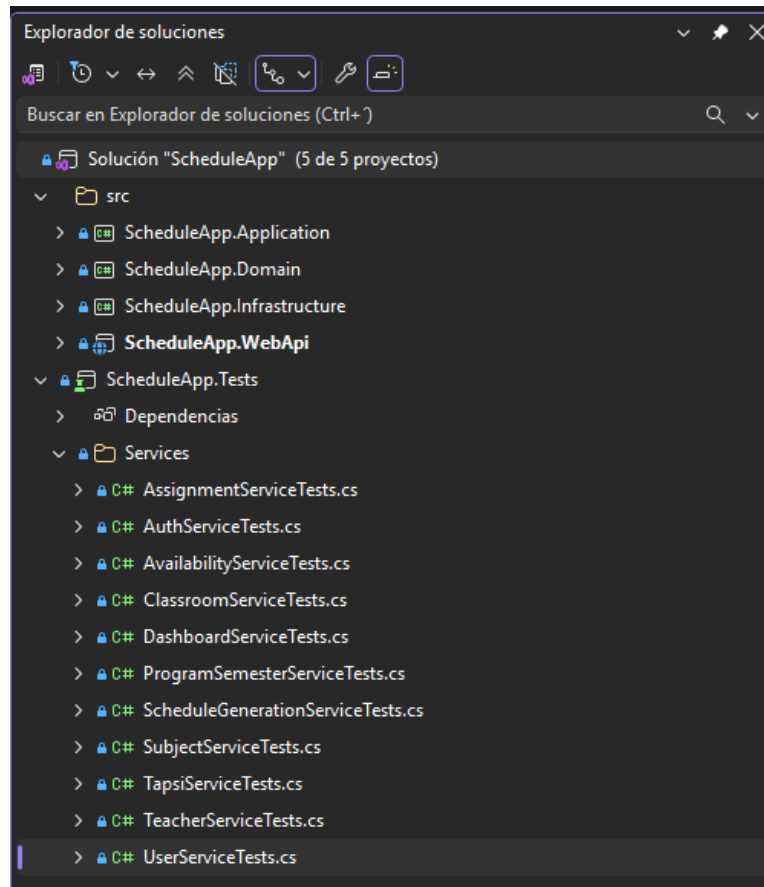
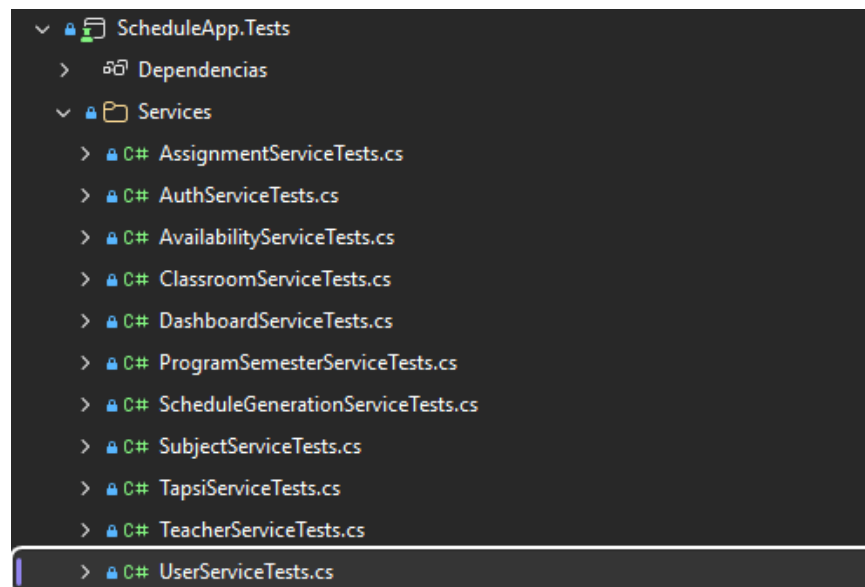


Pruebas Unitarias Backend

Estructura del proyecto de pruebas



Captura de archivos de pruebas unitarias creados



Captura de codigo de pruebas unitarias

AuthServiceTests

```
AuthServiceTests.cs
ScheduleApp.Tests

1 using FluentAssertions;
2 using Moq;
3 using ScheduleApp.Application.DTOs;
4 using ScheduleApp.Application.Interfaces;
5 using ScheduleApp.Application.Services;
6 using ScheduleApp.Domain.Entities;
7
8 namespace ScheduleApp.Tests.Services;
9
10 // Autor: Jacobo
11 // Version: 0.1
12 // Pruebas unitarias del AuthService (HU-167).
13 // Cubren los 4 casos de éxito y los 4 casos de error definidos en la HU.
14 // Se mockean IUserRepository, IPasswordHasher e IJwtService.
15 [Fact]
16 public class AuthServiceTests
17 {
18     private readonly Mock<IUserRepository> _userRepositoryMock;
19     private readonly Mock<IPasswordHasher> _passwordHasherMock;
20     private readonly Mock<IJwtService> _jwtMock;
21     private readonly AuthService _service;
22
23     public AuthServiceTests()
24     {
25         _userRepositoryMock = new Mock<IUserRepository>();
26         _passwordHasherMock = new Mock<IPasswordHasher>();
27         _jwtMock = new Mock<IJwtService>();
28         _service = new AuthService(
29             _userRepositoryMock.Object,
30             _passwordHasherMock.Object,
31             _jwtMock.Object);
32     }
33
34     // =====
35     // CASOS DE ÉXITO (4)
36     // =====
37
38     // CDA 1: Usuario con credenciales correctas puede iniciar sesión.
39     [Fact]
40     public async Task LoginAsync_WithValidCredentials_ShouldReturnLoginResponse()
41     {
42         var request = new LoginRequestDto
43         {
44             Login = "admin@uam.edu.co",
45             Password = "Password123"
46         };
47         var user = BuildActiveUser();
48         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
49             .ReturnsAsync(user);
50         _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
51             .Returns(true);
52         _jwtMock.Setup(j => j.GenerateToken(user)).Returns("fake-jwt-token");
53         _jwtMock.Setup(j => j.GetExpiration()).Returns(DateTime.UtcNow.AddMinutes(60));
54         var result = await _service.LoginAsync(request);
55         result.Should().NotBeNull();
56     }
57
58     // CDA 2: El servicio retorna una respuesta de autenticación válida (bien formada).
59     [Fact]
60     public async Task LoginAsync_WhenSuccessful_ShouldReturnNonNullValidResponse()
61     {
62         var request = BuildValidRequest();
63         var user = BuildActiveUser();
64         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
65             .ReturnsAsync(user);
66         _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
67             .Returns(true);
68         _jwtMock.Setup(j => j.GenerateToken(user)).Returns("fake-jwt-token");
69         _jwtMock.Setup(j => j.GetExpiration()).Returns(DateTime.UtcNow.AddMinutes(60));
70         var result = await _service.LoginAsync(request);
71         result.Should().NotBeNull();
72     }
73
74     // CDA 3: Se genera correctamente la información de sesión / token.
75     [Fact]
76     public async Task LoginAsync_WhenSuccessful_ShouldReturnAccessTokenAndExpiration()
77     {
78         var request = BuildValidRequest();
79         var user = BuildActiveUser();
80         var expectedToken = "generated-jwt-12345";
81         var expectedExpiration = DateTime.UtcNow.AddMinutes(60);
82         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
83             .ReturnsAsync(user);
84         _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
85             .Returns(true);
86         _jwtMock.Setup(j => j.GenerateToken(user)).Returns(expectedToken);
87         _jwtMock.Setup(j => j.GetExpiration()).Returns(expectedExpiration);
88         var result = await _service.LoginAsync(request);
89         result.AccessToken.Should().Be(expectedToken);
90         result.ExpiresAt.Should().Be(expectedExpiration);
91         _jwtMock.Verify(j => j.GenerateToken(user), Times.Once);
92     }
93
94     // CDA 4: Se retornan los datos básicos del usuario autenticado.
95     [Fact]
96     public async Task LoginAsync_WhenSuccessful_ShouldReturnUsernameAndRole()
97     {
98         var request = BuildValidRequest();
99         var user = new User
100         {
101             Id = Guid.NewGuid(),
102             FullName = "Jacobo Henao",
103             Email = "jhenao@uam.edu.co",
104             Username = "jhenao",
105             PasswordHash = "hashed",
106             IsActive = true,
107             Role = new Role { Id = Guid.NewGuid(), Name = "Administrador" }
108         };
109         SetupSuccessfulLogin(user);
110         var result = await _service.LoginAsync(request);
111         result.Username.Should().Be("Jacobo Henao");
112         result.Role.Should().Be("Administrador");
113     }
114
115     // =====
116     // CASOS DE ERROR (4)
117     // =====
118
119     // Error 1: No se permite iniciar sesión con contraseña incorrecta.
120     [Fact]
121     public async Task LoginAsync_WithInvalidPassword_ShouldThrowUnauthorized()
122     {
123         var request = BuildValidRequest();
124         request.Password = "WrongPassword";
125         var user = BuildActiveUser();
126         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
127             .ReturnsAsync(user);
128         _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
129             .Returns(false);
130         Func<Task> act = async () => await _service.LoginAsync(request);
131         await act.Should().ThrowAsync<UnauthorizedAccessException>()
132             .WithMessage("Credenciales incorrectas");
133         _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
134     }
135
136     // Error 2: No se permite iniciar sesión con correo / usuario no registrado.
137     [Fact]
138     public async Task LoginAsync_WithUnregisteredLogin_ShouldThrowUnauthorized()
139     {
140         var request = new LoginRequestDto
141         {
142             Login = "nonexistent@uam.edu.co",
143             Password = "Password123"
144         };
145         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
146             .ReturnsAsync((User?)null);
147         Func<Task> act = async () => await _service.LoginAsync(request);
148         await act.Should().ThrowAsync<UnauthorizedAccessException>()
149             .WithMessage("Credenciales incorrectas");
150         _passwordHasherMock.Verify(h => h.Verify(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
151         _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
152     }
153
154     // Error 3: No se permite iniciar sesión con campos vacíos.
155     // El repo no encuentra usuario con login vacío y el service lanza Unauthorized.
156     [Fact]
157     public async Task LoginAsync_WithEmptyLogin_ShouldThrowUnauthorized()
158     {
159         var request = new LoginRequestDto
160         {
161             Login = string.Empty,
162             Password = string.Empty
163         };
164         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(string.Empty))
165             .ReturnsAsync((User?)null);
166         Func<Task> act = async () => await _service.LoginAsync(request);
167         await act.Should().ThrowAsync<UnauthorizedAccessException>()
168             .WithMessage("Credenciales incorrectas");
169         _passwordHasherMock.Verify(h => h.Verify(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
170         _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
171     }
172
173     // Error 4: El servicio controla errores del repositorio propagándolos al controlador.
174     [Fact]
175     public async Task LoginAsync_WhenRepositoryThrows_ShouldPropagateError()
176     {
177         var request = BuildValidRequest();
178         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
179             .ThrowsAsync(new InvalidOperationException("DB connection error"));
180         Func<Task> act = async () => await _service.LoginAsync(request);
181         await act.Should().ThrowAsync<InvalidOperationException>()
182             .WithMessage("DB connection error");
183     }
184
185     // =====
186     // HELPERS
187     // =====
188
189     private static LoginRequestDto BuildValidRequest()
190     {
191         return new LoginRequestDto
192         {
193             Login = "admin@uam.edu.co",
194             Password = "Password123"
195         };
196     }
197
198     private static User BuildActiveUser()
199     {
200         return new User
201         {
202             Id = Guid.NewGuid(),
203             FullName = "Test User",
204             Email = "admin@uam.edu.co",
205             Username = "admin",
206             PasswordHash = "hashed-stored-password",
207             IsActive = true,
208             Role = new Role { Id = Guid.NewGuid(), Name = "Coordinador" }
209         };
210     }
211
212     private void SetupSuccessfulLogin(User user)
213     {
214         _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
215             .ReturnsAsync(user);
216         _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
217             .Returns(true);
218         _jwtMock.Setup(j => j.GenerateToken(user)).Returns("fake-jwt-token");
219         _jwtMock.Setup(j => j.GetExpiration()).Returns(DateTime.UtcNow.AddMinutes(60));
220     }
221 }
```

```
AuthServiceTests.cs
ScheduleApp.Tests

64 var user = BuildActiveUser();
65 SetupSuccessfulLogin(user);
66
67 var result = await _service.LoginAsync(request);
68
69 result.Should().NotBeNull();
70 result.AccessToken.Should().NotBeNullOrWhiteSpace();
71 result.Username.Should().NotBeNullOrWhiteSpace();
72 result.Role.Should().NotBeNullOrWhiteSpace();
73 result.ExpiresAt.Should().BeAfter(DateTime.UtcNow);
74
75 // CDA 3: Se genera correctamente la información de sesión / token.
76 [Fact]
77 public async Task LoginAsync_WhenSuccessful_ShouldReturnAccessTokenAndExpiration()
78 {
79     var request = BuildValidRequest();
80     var user = BuildActiveUser();
81     var expectedToken = "generated-jwt-12345";
82     var expectedExpiration = DateTime.UtcNow.AddMinutes(60);
83     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
84         .ReturnsAsync(user);
85     _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
86         .Returns(true);
87     _jwtMock.Setup(j => j.GenerateToken(user)).Returns(expectedToken);
88     _jwtMock.Setup(j => j.GetExpiration()).Returns(expectedExpiration);
89     var result = await _service.LoginAsync(request);
90     result.AccessToken.Should().Be(expectedToken);
91     result.ExpiresAt.Should().Be(expectedExpiration);
92     _jwtMock.Verify(j => j.GenerateToken(user), Times.Once);
93 }
94
95 // CDA 4: Se retornan los datos básicos del usuario autenticado.
96 [Fact]
97 public async Task LoginAsync_WhenSuccessful_ShouldReturnUsernameAndRole()
98 {
99     var request = BuildValidRequest();
100     var user = new User
101     {
102         Id = Guid.NewGuid(),
103         FullName = "Jacobo Henao",
104         Email = "jhenao@uam.edu.co",
105         Username = "jhenao",
106         PasswordHash = "hashed",
107         IsActive = true,
108         Role = new Role { Id = Guid.NewGuid(), Name = "Administrador" }
109     };
110     SetupSuccessfulLogin(user);
111     var result = await _service.LoginAsync(request);
112     result.Username.Should().Be("Jacobo Henao");
113     result.Role.Should().Be("Administrador");
114 }
115
116 // =====
117 // CASOS DE ERROR (4)
118 // =====
119
120 // Error 1: No se permite iniciar sesión con contraseña incorrecta.
121 [Fact]
122 public async Task LoginAsync_WithInvalidPassword_ShouldThrowUnauthorized()
123 {
124     var request = BuildValidRequest();
125     request.Password = "WrongPassword";
126     var user = BuildActiveUser();
127     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
128         .ReturnsAsync(user);
129     _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
130         .Returns(false);
131     Func<Task> act = async () => await _service.LoginAsync(request);
132     await act.Should().ThrowAsync<UnauthorizedAccessException>()
133         .WithMessage("Credenciales incorrectas");
134     _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
135 }
136
137 // Error 2: No se permite iniciar sesión con correo / usuario no registrado.
138 [Fact]
139 public async Task LoginAsync_WithUnregisteredLogin_ShouldThrowUnauthorized()
140 {
141     var request = new LoginRequestDto
142     {
143         Login = "nonexistent@uam.edu.co",
144         Password = "Password123"
145     };
146     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
147         .ReturnsAsync((User?)null);
148     Func<Task> act = async () => await _service.LoginAsync(request);
149     await act.Should().ThrowAsync<UnauthorizedAccessException>()
150         .WithMessage("Credenciales incorrectas");
151     _passwordHasherMock.Verify(h => h.Verify(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
152     _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
153 }
154
155 // Error 3: No se permite iniciar sesión con campos vacíos.
156 // El repo no encuentra usuario con login vacío y el service lanza Unauthorized.
157 [Fact]
158 public async Task LoginAsync_WithEmptyLogin_ShouldThrowUnauthorized()
159 {
160     var request = new LoginRequestDto
161     {
162         Login = string.Empty,
163         Password = string.Empty
164     };
165     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(string.Empty))
166         .ReturnsAsync((User?)null);
167     Func<Task> act = async () => await _service.LoginAsync(request);
168     await act.Should().ThrowAsync<UnauthorizedAccessException>()
169         .WithMessage("Credenciales incorrectas");
170     _passwordHasherMock.Verify(h => h.Verify(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
171     _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
172 }
173
174 // Error 4: El servicio controla errores del repositorio propagándolos al controlador.
175 [Fact]
176 public async Task LoginAsync_WhenRepositoryThrows_ShouldPropagateError()
177 {
178     var request = BuildValidRequest();
179     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
180         .ThrowsAsync(new InvalidOperationException("DB connection error"));
181     Func<Task> act = async () => await _service.LoginAsync(request);
182     await act.Should().ThrowAsync<InvalidOperationException>()
183         .WithMessage("DB connection error");
184 }
185
186 // =====
187 // HELPERS
188 // =====
189
190 private static LoginRequestDto BuildValidRequest()
191 {
192     return new LoginRequestDto
193     {
194         Login = "admin@uam.edu.co",
195         Password = "Password123"
196     };
197 }
198
199 private static User BuildActiveUser()
200 {
201     return new User
202     {
203         Id = Guid.NewGuid(),
204         FullName = "Test User",
205         Email = "admin@uam.edu.co",
206         Username = "admin",
207         PasswordHash = "hashed-stored-password",
208         IsActive = true,
209         Role = new Role { Id = Guid.NewGuid(), Name = "Coordinador" }
210     };
211 }
212
213 private void SetupSuccessfulLogin(User user)
214 {
215     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
216         .ReturnsAsync(user);
217     _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
218         .Returns(true);
219     _jwtMock.Setup(j => j.GenerateToken(user)).Returns("fake-jwt-token");
220     _jwtMock.Setup(j => j.GetExpiration()).Returns(DateTime.UtcNow.AddMinutes(60));
221 }
222 }
```

```
AuthServiceTests.cs
ScheduleApp.Tests

128 {
129     var request = new LoginRequestDto
130     {
131         Login = "admin@uam.edu.co",
132         Password = "wrong-password"
133     };
134     var user = BuildActiveUser();
135     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
136         .ReturnsAsync(user);
137     _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
138         .Returns(false);
139     Func<Task> act = async () => await _service.LoginAsync(request);
140     await act.Should().ThrowAsync<UnauthorizedAccessException>()
141         .WithMessage("Credenciales incorrectas");
142     _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
143 }
144
145 // Error 2: No se permite iniciar sesión con correo / usuario no registrado.
146 [Fact]
147 public async Task LoginAsync_WithUnregisteredLogin_ShouldThrowUnauthorized()
148 {
149     var request = new LoginRequestDto
150     {
151         Login = "nonexistent@uam.edu.co",
152         Password = "Password123"
153     };
154     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
155         .ReturnsAsync((User?)null);
156     Func<Task> act = async () => await _service.LoginAsync(request);
157     await act.Should().ThrowAsync<UnauthorizedAccessException>()
158         .WithMessage("Credenciales incorrectas");
159     _passwordHasherMock.Verify(h => h.Verify(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
160     _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
161 }
162
163 // Error 3: No se permite iniciar sesión con campos vacíos.
164 // El repo no encuentra usuario con login vacío y el service lanza Unauthorized.
165 [Fact]
166 public async Task LoginAsync_WithEmptyLogin_ShouldThrowUnauthorized()
167 {
168     var request = new LoginRequestDto
169     {
170         Login = string.Empty,
171         Password = string.Empty
172     };
173     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(string.Empty))
174         .ReturnsAsync((User?)null);
175     Func<Task> act = async () => await _service.LoginAsync(request);
176     await act.Should().ThrowAsync<UnauthorizedAccessException>()
177         .WithMessage("Credenciales incorrectas");
178     _passwordHasherMock.Verify(h => h.Verify(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
179     _jwtMock.Verify(j => j.GenerateToken(It.IsAny<User>()), Times.Never);
180 }
181
182 // Error 4: El servicio controla errores del repositorio propagándolos al controlador.
183 [Fact]
184 public async Task LoginAsync_WhenRepositoryThrows_ShouldPropagateError()
185 {
186     var request = BuildValidRequest();
187     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
188         .ThrowsAsync(new InvalidOperationException("DB connection error"));
189     Func<Task> act = async () => await _service.LoginAsync(request);
190     await act.Should().ThrowAsync<InvalidOperationException>()
191         .WithMessage("DB connection error");
192 }
193
194 // =====
195 // HELPERS
196 // =====
197
198 private static LoginRequestDto BuildValidRequest()
199 {
200     return new LoginRequestDto
201     {
202         Login = "admin@uam.edu.co",
203         Password = "Password123"
204     };
205 }
206
207 private static User BuildActiveUser()
208 {
209     return new User
210     {
211         Id = Guid.NewGuid(),
212         FullName = "Test User",
213         Email = "admin@uam.edu.co",
214         Username = "admin",
215         PasswordHash = "hashed-stored-password",
216         IsActive = true,
217         Role = new Role { Id = Guid.NewGuid(), Name = "Coordinador" }
218     };
219 }
220
221 private void SetupSuccessfulLogin(User user)
222 {
223     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
224         .ReturnsAsync(user);
225     _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
226         .Returns(true);
227     _jwtMock.Setup(j => j.GenerateToken(user)).Returns("fake-jwt-token");
228     _jwtMock.Setup(j => j.GetExpiration()).Returns(DateTime.UtcNow.AddMinutes(60));
229 }
230 }
```

```
AuthServiceTests.cs
ScheduleApp.Tests

190 {
191     var request = BuildValidRequest();
192     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
193         .ThrowsAsync(new InvalidOperationException("DB connection error"));
194     Func<Task> act = async () => await _service.LoginAsync(request);
195     await act.Should().ThrowAsync<InvalidOperationException>()
196         .WithMessage("DB connection error");
197 }
198
199 // =====
200 // HELPERS
201 // =====
202
203 private static LoginRequestDto BuildValidRequest()
204 {
205     return new LoginRequestDto
206     {
207         Login = "admin@uam.edu.co",
208         Password = "Password123"
209     };
210 }
211
212 private static User BuildActiveUser()
213 {
214     return new User
215     {
216         Id = Guid.NewGuid(),
217         FullName = "Test User",
218         Email = "admin@uam.edu.co",
219         Username = "admin",
220         PasswordHash = "hashed-stored-password",
221         IsActive = true,
222         Role = new Role { Id = Guid.NewGuid(), Name = "Coordinador" }
223     };
224 }
225
226 private void SetupSuccessfulLogin(User user)
227 {
228     _userRepositoryMock.Setup(r => r.GetByEmailOrUsernameAsync(request.Login))
229         .ReturnsAsync(user);
230     _passwordHasherMock.Setup(h => h.Verify(request.Password, user.PasswordHash))
231         .Returns(true);
232     _jwtMock.Setup(j => j.GenerateToken(user)).Returns("fake-jwt-token");
233     _jwtMock.Setup(j => j.GetExpiration()).Returns(DateTime.UtcNow.AddMinutes(60));
234 }
235 }
```

ClassroomServiceTests

```
ClassroomServiceTests.cs
ScheduleApp.Tests
1 using FluentAssertions;
2 using Moq;
3 using ScheduleApp.Application.Interfaces;
4 using ScheduleApp.Application.Services;
5 using ScheduleApp.Domain.Entities;
6
7 namespace ScheduleApp.Tests.Services
8 {
9     [TestFixture]
10     public class ClassroomServiceTests
11     {
12         private readonly Mock<IClassroomRepository> _repositoryMock;
13         private readonly ClassroomService _service;
14
15         public ClassroomServiceTests()
16         {
17             _repositoryMock = new Mock<IClassroomRepository>();
18             _service = new ClassroomService(_repositoryMock.Object);
19         }
20
21         [Fact]
22         public async Task CreateClassroomAsync_ShouldCreateClassroom_WhenDataIsValid()
23         {
24             // Arrange
25             var classroom = new Classroom
26             {
27                 Id = Guid.NewGuid(),
28                 Code = "A101",
29                 Name = "Sala 101",
30                 Building = "B1",
31                 Floor = 1,
32                 Capacity = 20,
33                 Type = "Teórica",
34                 IsActive = true
35             };
36
37             _repositoryMock
38                 .Setup(r => r.GetByIdAsync(classroom.Code))
39                 .ReturnsAsync((Classroom?)null);
40
41             // Act
42             await _service.CreateClassroomAsync(classroom);
43
44             // Assert
45             _repositoryMock.Verify(r =>
46                 r.CreateAsync(it => it.IsActive == true, Times.Once);
47             );
48         }
49
50         [Fact]
51         public async Task CreateClassroomAsync_ShouldThrowException_WhenCodeAlreadyExists()
52         {
53             // Arrange
54             var classroom = new Classroom
55             {
56                 Id = Guid.NewGuid(),
57                 Code = "A101",
58                 Name = "Sala Duplicada"
59             };
60
61             _repositoryMock
62                 .Setup(r => r.GetByIdAsync(classroom.Code))
63                 .ReturnsAsync(classroom);
64
65             // Act
66             Func<Task> action = async () =>
67                 await _service.CreateClassroomAsync(classroom);
68
69             // Assert
70             await action.Should().ThrowAsync<InvalidOperationException>();
71         }
72
73         [Fact]
74         public async Task GetClassroomsAsync_ShouldReturnClassroomList()
75         {
76             // Arrange
77             var classrooms = new List<Classroom>
78             {
79                 new Classroom
80                 {
81                     Id = Guid.NewGuid(),
82                     Code = "A101",
83                     Name = "Sala 101",
84                     Building = "B1",
85                     Floor = 1,
86                     Capacity = 20,
87                     Type = "Teórica",
88                     IsActive = true
89                 }
90             };
91
92             _repositoryMock
93                 .Setup(r => r.GetAllAsync())
94                 .ReturnsAsync(classrooms);
95
96             // Act
97             var result = await _service.GetClassroomsAsync();
98
99             // Assert
100             result.Should().NotBeNull();
101             result.Count.Should().Be(1);
102         }
103
104         [Fact]
105         public async Task UpdateClassroomAsync_ShouldUpdateClassroom_WhenValid()
106         {
107             // Arrange
108             var classroom = new Classroom
109             {
110                 Id = Guid.NewGuid(),
111                 Code = "A101",
112                 Name = "Sala Actualizada"
113             };
114
115             _repositoryMock
116                 .Setup(r => r.GetByIdAsync(classroom.Code))
117                 .ReturnsAsync(classroom);
118
119             // Act
120             await _service.UpdateClassroomAsync(classroom);
121
122             // Assert
123             _repositoryMock.Verify(r =>
124                 r.UpdateAsync(it => it.Name == "Sala Actualizada", Times.Once);
125             );
126         }
127
128         [Fact]
129         public async Task DeleteClassroomAsync_ShouldDeleteClassroom()
130         {
131             // Arrange
132             var id = Guid.NewGuid();
133
134             _repositoryMock
135                 .Setup(r => r.GetByIdAsync(id))
136                 .ReturnsAsync((Classroom?)null);
137
138             // Act
139             await _service.DeleteClassroomAsync(id);
140
141             // Assert
142             _repositoryMock.Verify(r =>
143                 r.DeleteAsync(id, Times.Once);
144             );
145         }
146     }
147 }
```

```
ClassroomServiceTests.cs
ScheduleApp.Tests
104
105     [Fact]
106     public async Task GetClassroomByIdAsync_ShouldReturnClassroom_WhenExists()
107     {
108         // Arrange
109         var id = Guid.NewGuid();
110
111         var classroom = new Classroom
112         {
113             Id = id,
114             Code = "A101",
115             Name = "Sala 101"
116         };
117
118         _repositoryMock
119             .Setup(r => r.GetByIdAsync(id))
120             .ReturnsAsync(classroom);
121
122         // Act
123         var result = await _service.GetClassroomByIdAsync(id);
124
125         // Assert
126         result.Should().NotBeNull();
127         result.Id.Should().Be(id);
128     }
129
130     [Fact]
131     public async Task UpdateClassroomAsync_ShouldUpdateClassroom_WhenValid()
132     {
133         // Arrange
134         var classroom = new Classroom
135         {
136             Id = Guid.NewGuid(),
137             Code = "A101",
138             Name = "Sala Actualizada"
139         };
140
141         _repositoryMock
142             .Setup(r => r.GetByIdAsync(classroom.Code))
143             .ReturnsAsync(classroom);
144
145         // Act
146         await _service.UpdateClassroomAsync(classroom);
147
148         // Assert
149         _repositoryMock.Verify(r =>
150             r.UpdateAsync(it => it.Name == "Sala Actualizada", Times.Once);
151         );
152     }
153
154     [Fact]
155     public async Task ChangeStatusAsync_ShouldChangeClassroomStatus()
156     {
157         // Arrange
158         var id = Guid.NewGuid();
159
160         var classroom = new Classroom
161         {
162             Id = id,
163             Code = "A101",
164             Name = "Sala 101",
165             IsActive = true
166         };
167
168         _repositoryMock
169             .Setup(r => r.GetByIdAsync(id))
170             .ReturnsAsync(classroom);
171
172         // Act
173         var result = await _service.ChangeStatusAsync(id, false);
174
175         // Assert
176         result.Should().NotBeNull();
177         result.IsActive.Should().BeFalse();
178
179         _repositoryMock.Verify(r =>
180             r.UpdateAsync(it => it.IsActive == false, Times.Once);
181         );
182     }
183
184     [Fact]
185     public async Task ChangeStatusAsync_ShouldReturnNull_WhenClassroomDoesNotExist()
186     {
187         // Arrange
188         var id = Guid.NewGuid();
189
190         _repositoryMock
191             .Setup(r => r.GetByIdAsync(id))
192             .ReturnsAsync((Classroom?)null);
193
194         // Act
195         var result = await _service.ChangeStatusAsync(id, false);
196
197         // Assert
198         result.Should().BeNull();
199     }
200
201     [Fact]
202     public async Task DeleteClassroomAsync_ShouldCallDeleteAsync()
203     {
204         // Arrange
205         var id = Guid.NewGuid();
206
207         // Act
208         await _service.DeleteClassroomAsync(id);
209
210         // Assert
211         _repositoryMock.Verify(r =>
212             r.DeleteAsync(id, Times.Once);
213         );
214     }
215 }
```

```
ClassroomServiceTests.cs
ScheduleApp.Tests
146
147     [Fact]
148     public async Task UpdateClassroomAsync_ShouldUpdateClassroom_WhenValid()
149     {
150         // Arrange
151         var classroom = new Classroom
152         {
153             Id = Guid.NewGuid(),
154             Code = "A101",
155             Name = "Sala Actualizada"
156         };
157
158         _repositoryMock
159             .Setup(r => r.GetByIdAsync(classroom.Code))
160             .ReturnsAsync(classroom);
161
162         // Act
163         await _service.UpdateClassroomAsync(classroom);
164
165         // Assert
166         _repositoryMock.Verify(r =>
167             r.UpdateAsync(it => it.Name == "Sala Actualizada", Times.Once);
168         );
169     }
170
171     [Fact]
172     public async Task DeleteClassroomAsync_ShouldCallDeleteAsync()
173     {
174         // Arrange
175         var id = Guid.NewGuid();
176
177         _repositoryMock
178             .Setup(r => r.GetByIdAsync(id))
179             .ReturnsAsync((Classroom?)null);
180
181         // Act
182         await _service.DeleteClassroomAsync(id);
183
184         // Assert
185         _repositoryMock.Verify(r =>
186             r.DeleteAsync(id, Times.Once);
187         );
188     }
189
190     [Fact]
191     public async Task ChangeStatusAsync_ShouldReturnNull_WhenClassroomDoesNotExist()
192     {
193         // Arrange
194         var id = Guid.NewGuid();
195
196         _repositoryMock
197             .Setup(r => r.GetByIdAsync(id))
198             .ReturnsAsync((Classroom?)null);
199
200         // Act
201         var result = await _service.ChangeStatusAsync(id, false);
202
203         // Assert
204         result.Should().BeNull();
205     }
206
207     [Fact]
208     public async Task DeleteClassroomAsync_ShouldCallDeleteAsync()
209     {
210         // Arrange
211         var id = Guid.NewGuid();
212
213         _repositoryMock
214             .Setup(r => r.GetByIdAsync(id))
215             .ReturnsAsync((Classroom?)null);
216
217         // Act
218         await _service.DeleteClassroomAsync(id);
219
220         // Assert
221         _repositoryMock.Verify(r =>
222             r.DeleteAsync(id, Times.Once);
223         );
224     }
225 }
```

DashboardServiceTest

```
DashboardServiceTests.cs
ScheduleApp.Tests
ScheduleApp.Tests

1 using FluentAssertions;
2 using Moq;
3 using ScheduleApp.Application.DTOS;
4 using ScheduleApp.Application.Interfaces;
5 using ScheduleApp.Application.Services;
6
7 namespace ScheduleApp.Tests.Services
8 {
9     // Hotfix: el constructor de DashboardService cambio para requerir IDashboardRepository.
10    // Se agrega el mock del repositorio para que los tests compilen.
11    1 referencia
12    public class DashboardServiceTests
13    {
14        private readonly Mock<IDashboardRepository> _repoMock;
15        private readonly DashboardService _service;
16
17        0 referencias
18        public DashboardServiceTests()
19        {
20            _repoMock = new Mock<IDashboardRepository>();
21            _repoMock.Setup(r => r.GetSummary()).Returns(new DashboardSummaryDto
22            {
23                Subjects = 5,
24                Teachers = 3,
25                Schedules = 7,
26                Programs = 2,
27                Classrooms = 4,
28                Coordinators = 1
29            });
30            _service = new DashboardService(_repoMock.Object);
31        }
32
33        0 referencias
34        [Fact]
35        public void GetSummary_DeberiaRetornarResumenNoNulo()
36        {
37            // Act
38            var result = _service.GetSummary();
39
40            // Assert
41            result.Should().NotBeNull();
42        }
43
44        0 referencias
45        [Fact]
46        public void GetSummary_DeberiaRetornarValoresMayoresOigualesACero()
47        {
48            // Act
49            var result = _service.GetSummary();
50
51            // Assert
52            result.Subjects.Should().BeGreaterThanOrEqualTo(0);
53            result.Teachers.Should().BeGreaterThanOrEqualTo(0);
54            result.Schedules.Should().BeGreaterThanOrEqualTo(0);
55            result.Programs.Should().BeGreaterThanOrEqualTo(0);
56            result.Classrooms.Should().BeGreaterThanOrEqualTo(0);
57            result.Coordinators.Should().BeGreaterThanOrEqualTo(0);
58        }
59    }
60 }
```

ScheduleGenerationServiceTests

```
ScheduleGene...viceTests.cs
ScheduleApp.Tests
1 using FluentAssertions;
2 using Moq;
3 using ScheduleApp.Application.DTOs;
4 using ScheduleApp.Application.Interfaces;
5 using ScheduleApp.Services;
6 namespace ScheduleApp.Tests.Services;
7
8 // Autor: Jacobo
9 // Version: 3.1
10 // Pruebas unitarias del ScheduleGenerationService (M0169).
11 // Caneva los 7 casos de éxito y los 6 casos de error definidos en la MD.
12 // El servicio refactorizado por el equipo delega la lógica de generación al
13 // ScheduleRepository, por lo que las validaciones de esta, cruces y reglas
14 // especiales se prueban verificando que el servicio delega correctamente al
15 // repository (que es responsable de aplicar esas reglas).
16
17 public class ScheduleGenerationServiceTests
18 {
19     private readonly Mock<IScheduleRepository> _scheduleRepoMock;
20     private readonly ScheduleGenerationService _service;
21
22     public ScheduleGenerationServiceTests()
23     {
24         _scheduleRepoMock = new Mock<IScheduleRepository>();
25         _service = new ScheduleGenerationService(_scheduleRepoMock.Object);
26
27         // =====
28         // CASOS DE ÉXITO (7)
29         // =====
30
31         // COM 1: Generar horario con datos válidos.
32         [Fact]
33         public async Task GenerateAsync_WithValidData_ShouldReturnSuccessResponse()
34         {
35             var request = BuildValidRequest();
36             var schedules = BuildSampleSchedules();
37             _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
38                 .ReturnsAsync(true);
39             _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
40                 request.AcademicProgramId, request.SemesterNumber, request.Shift))
41                 .ReturnsAsync(schedules);
42
43             var result = await _service.GenerateAsync(request);
44
45             result.Should().NotBeNull();
46             result.Success.Should().BeTrue();
47             result.Message.Should().Contain("correctamente");
48         }
49
50         // COM 2: Los horarios generados se retornan listos para guardar.
51         [Fact]
52         public async Task GenerateAsync_WhenSchedulesGenerated_ShouldReturnValidResponse()
53         {
54             var request = BuildValidRequest();
55             var schedules = BuildSampleSchedules();
56             _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
57                 .ReturnsAsync(true);
58             _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
59                 request.AcademicProgramId, request.SemesterNumber, request.Shift))
60                 .ReturnsAsync(schedules);
61
62             var result = await _service.GenerateAsync(request);
63
64             result.GeneratedSchedules.Should().HaveCount(3);
65             result.GeneratedSchedules.Should().BeEquivalentTo(schedules);
66         }
67
68         // COM 3: La respuesta trae la metadata correcta de cantidad de materias.
69         [Fact]
70         public async Task GenerateAsync_ShouldReturnCorrectMetadata()
71         {
72             var request = BuildValidRequest();
73             var schedules = BuildSampleSchedules();
74             _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
75                 .ReturnsAsync(true);
76             _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
77                 request.AcademicProgramId, request.SemesterNumber, request.Shift))
78                 .ReturnsAsync(schedules);
79
80             var result = await _service.GenerateAsync(request);
81
82             result.GeneratedSchedules.Count.Should().Be(3);
83             result.GeneratedSchedules.Count.Should().Be(3);
84
85             // =====
86             // CASOS DE ERROR (6)
87             // =====
88
89             // Error 1: No se genera horario sin AcademicProgramId.
90             [Fact]
91             public async Task GenerateAsync_WithEmptyProgramId_ShouldReturnError()
92             {
93                 var request = new GenerateScheduleRequestDto
94                 {
95                     AcademicProgramId = Guid.Empty,
96                     SemesterNumber = 1,
97                     Shift = "Mañana",
98                 };
99
100                 var result = await _service.GenerateAsync(request);
101
102                 result.Success.Should().BeFalse();
103                 result.Message.Should().Contain("programa");
104                 _scheduleRepoMock.Verify(r => r.GetSubjectsForGenerationAsync(
105                     request.AcademicProgramId, request.SemesterNumber, request.Shift));
106             }
107
108             // Error 2 (continuación): No se genera con SemesterNumber invalido.
109             [Fact]
110             public async Task GenerateAsync_WithInvalidSemester_ShouldReturnError()
111             {
112                 var request = new GenerateScheduleRequestDto
113                 {
114                     AcademicProgramId = Guid.NewGuid(),
115                     SemesterNumber = 0,
116                     Shift = "Mañana",
117                 };
118
119                 var result = await _service.GenerateAsync(request);
120
121                 result.Success.Should().BeFalse();
122                 result.Message.Should().Contain("semestre");
123             }
124
125             // Error 3 (continuación): No se genera sin Shift.
126             [Fact]
127             public async Task GenerateAsync_WithEmptyShift_ShouldReturnError()
128             {
129                 var request = new GenerateScheduleRequestDto
130                 {
131                     AcademicProgramId = Guid.NewGuid(),
132                     SemesterNumber = 1,
133                     Shift = "",
134                 };
135
136                 var result = await _service.GenerateAsync(request);
137
138                 result.Success.Should().BeFalse();
139                 result.Message.Should().Contain("horario");
140             }
141
142             // Error 4: Cuando el programa no exista en BD se controla con mensaje claro.
143             [Fact]
144             public async Task GenerateAsync_WhenProgramNotExists_ShouldReturnError()
145             {
146                 var request = BuildValidRequest();
147                 _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
148                     .ReturnsAsync(false);
149
150                 var result = await _service.GenerateAsync(request);
151
152                 result.Success.Should().BeFalse();
153                 result.Message.Should().Contain("programa no existe");
154             }
155
156             // Error 5: Si el repository lanza, el service propaga la excepción al controller.
157             [Fact]
158             public async Task GenerateAsync_WhenRepositoryThrows_ShouldPropagateError()
159             {
160                 var request = BuildValidRequest();
161                 _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
162                     .ReturnsAsync(true);
163                 _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
164                     request.AcademicProgramId, request.SemesterNumber, request.Shift))
165                     .ThrowsAsync(new InvalidOperationException("DB connection error"));
166
167                 Func<Task> act = async () => await _service.GenerateAsync(request);
168                 await act.Should().ThrowAsync<InvalidOperationException>();
169             }
170
171             // Error 6: Cuando no es posible generar (sin materias) la respuesta es controlada.
172             [Fact]
173             public async Task GenerateAsync_WhenNoSubjectsFound_ShouldReturnErrorWithWarning()
174             {
175                 var request = BuildValidRequest();
176                 _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
177                     .ReturnsAsync(true);
178                 _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
179                     request.AcademicProgramId, request.SemesterNumber, request.Shift))
180                     .ReturnsAsync(new List<GeneratedScheduleEntryDto>());
181
182                 var result = await _service.GenerateAsync(request);
183
184                 result.Success.Should().BeFalse();
185                 result.Message.Should().NotBeNullOrEmpty();
186                 result.Warnings.Should().NotBeEmpty();
187                 result.GeneratedSchedules.Should().BeEmpty();
188             }
189
190             // =====
191             // HELPERS
192             // =====
193
194             private static GenerateScheduleRequestDto BuildValidRequest()
195             {
196                 return new GenerateScheduleRequestDto
197                 {
198                     AcademicProgramId = Guid.NewGuid(),
199                     SemesterNumber = 1,
200                     Shift = "Mañana",
201                 };
202             }
203
204             private static List<GeneratedScheduleEntryDto> BuildSampleSchedules(int count)
205             {
206                 var schedules = new List<GeneratedScheduleEntryDto>();
207                 for (int i = 0; i < count; i++)
208                 {
209                     schedules.Add(new GeneratedScheduleEntryDto
210                     {
211                         Id = Guid.NewGuid(),
212                     });
213                 }
214                 return schedules;
215             }
216         }
217     }
218
219     ScheduleApp.Tests
220     1 _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
221         .ReturnsAsync(false);
222     2 var result = await _service.GenerateAsync(request);
223     3
224     4 result.Success.Should().BeFalse();
225     5 result.Message.Should().Contain("programa no existe");
226     6 _scheduleRepoMock.Verify(r => r.GetSubjectsForGenerationAsync(
227         request.AcademicProgramId, request.SemesterNumber, request.Shift));
228     7
229     8 // Error 8: Si el repository lanza, el service propaga la excepción al controller.
230     9 [Fact]
231    10 public async Task GenerateAsync_WhenRepositoryThrows_ShouldPropagateError()
232    11 {
233    12     var request = BuildValidRequest();
234    13     _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
235    14         .ReturnsAsync(true);
236    15     _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
237         request.AcademicProgramId, request.SemesterNumber, request.Shift))
238    16         .ThrowsAsync(new InvalidOperationException("DB connection error"));
239    17
240    18     Func<Task> act = async () => await _service.GenerateAsync(request);
241    19     await act.Should().ThrowAsync<InvalidOperationException>();
242    20
243    21 // Error 9: Cuando no es posible generar (sin materias) la respuesta es controlada.
244    22 [Fact]
245    23 public async Task GenerateAsync_WhenNoSubjectsFound_ShouldReturnErrorWithWarning()
246    24 {
247    25     var request = BuildValidRequest();
248    26     _scheduleRepoMock.Setup(r => r.AcademicProgramExistsAsync(request.AcademicProgramId))
249    27         .ReturnsAsync(true);
250    28     _scheduleRepoMock.Setup(r => r.GetSubjectsForGenerationAsync(
251         request.AcademicProgramId, request.SemesterNumber, request.Shift))
252    29         .ReturnsAsync(new List<GeneratedScheduleEntryDto>());
253    30
254    31     var result = await _service.GenerateAsync(request);
255    32
256    33     result.Success.Should().BeFalse();
257    34     result.Message.Should().NotBeNullOrEmpty();
258    35     result.Warnings.Should().NotBeEmpty();
259    36     result.GeneratedSchedules.Should().BeEmpty();
260    37
261    38 // =====
262    39 // HELPERS
263    40 // =====
264
265    41 private static GenerateScheduleRequestDto BuildValidRequest()
266    42 {
267    43     return new GenerateScheduleRequestDto
268    44     {
269    45         AcademicProgramId = Guid.NewGuid(),
270    46         SemesterNumber = 1,
271    47         Shift = "Mañana",
272    48     };
273    49
274    50 private static List<GeneratedScheduleEntryDto> BuildSampleSchedules(int count)
275    51 {
276    52     var schedules = new List<GeneratedScheduleEntryDto>();
277    53     for (int i = 0; i < count; i++)
278    54     {
279    55         schedules.Add(new GeneratedScheduleEntryDto
280    56         {
281    57             Id = Guid.NewGuid(),
282    58         });
283    59     }
284    60     return schedules;
285    61 }
286 }
```

```

1  using FluentAssertions;
2  using Moq;
3  using ScheduleApp.Application.DTOS;
4  using ScheduleApp.Application.Interfaces;
5  using ScheduleApp.Application.Services;
6  using ScheduleApp.Domain.Entities;
7
8  namespace ScheduleApp.Tests.Services
9  {
10     [TestFixture]
11     public class SubjectServiceTests
12     {
13         private readonly Mock<ISubjectRepository> _repoMock;
14         private readonly SubjectService _service;
15
16         [SetUp]
17         public SubjectServiceTests()
18         {
19             _repoMock = new Mock<ISubjectRepository>();
20             _service = new SubjectService(_repoMock.Object);
21         }
22
23         [Fact]
24         public async Task CreateSubjectAsync_DatabaseCreateFailsCorrectly()
25         {
26             // Arrange
27             var dto = new CreateSubjectDto
28             {
29                 Code = "MAT101",
30                 Name = "Matemáticas",
31                 Semester = 3,
32                 Credits = 3,
33                 WeeklyHours = 4
34             };
35
36             _repoMock.Setup(r => r.CreateCodeAsync(dto.Code))
37                 .ReturnsAsync((Subject?)null);
38
39             // Act
40             await _service.CreateSubjectAsync(dto);
41
42             // Assert
43             _repoMock.Verify(r =>
44                 r.CreateAsync(It.IsAny<Subject>()),
45                 Times.Once);
46         }
47
48         [Fact]
49         public async Task CreateSubjectAsync_SubjectAlreadyExistsDuplicated()
50         {
51             // Arrange
52             var dto = new CreateSubjectDto
53             {
54                 Code = "MAT101",
55                 Name = "Matemáticas",
56                 Semester = 3,
57                 Credits = 3,
58                 WeeklyHours = 4
59             };
60
61             _repoMock.Setup(r => r.CreateCodeAsync(dto.Code))
62                 .ReturnsAsync(new Subject());
63
64             // Act
65             Func<Task> act = async () =>
66                 await _service.CreateSubjectAsync(dto);
67
68             // Assert
69             await act.Should()
70                 .ThrowAsync<Exception>()
71                 .WithMessage("Subject code already exists");
72         }
73
74         [Fact]
75         public async Task CreateSubjectAsync_SubjectAlreadyExistsValid()
76         {
77             // Arrange
78             var dto = new CreateSubjectDto
79             {
80                 Code = "",

```

```

137 // Act
138 await _service.UpdateSubjectKeync(subjectId, dto);
139
140 // Assert
141 _response.Verify(r =>
142     r.SubjectKeync(It.IsAny<SubjectKeync>()),
143     Times.Once);
144 }
145
146 [Fact]
147 public async Task UpdateSubjectKeync_DatabaseControlNotMaterializesItems()
148 {
149     // Arrange
150     var subjectId = Guid.NewGuid();
151     var dto = new UpdateSubjectDto
152     {
153         Code = "HIS",
154         Name = "Hesse",
155         Semester = 2,
156         Credits = 4,
157         WeeklyHours = 3,
158         IsActive = true
159     };
160
161     _response.Setup(r => r.GetSubjectKeync(subjectId))
162         .ReturnsAsync(SubjectKeync.Null);
163
164     // Act
165     Func<Task> act = async () =>
166         await _service.UpdateSubjectKeync(subjectId, dto);
167
168     // Assert
169     await act.Should()
170         .NotThrowAsync<Exception>()
171         .WithMessage("Subject not found");
172 }
173
174 [Fact]
175 public async Task DeleteSubjectKeync_DatabaseControlMaterializes()
176 {
177     // Arrange
178     var subjectId = Guid.NewGuid();
179     var subject = new Subject
180     {
181         Id = subjectId,
182         IsActive = true
183     };
184
185     _response.Setup(r => r.GetSubject(subjectId))
186         .ReturnsAsync(subject);
187
188     // Act
189     var result = await _service.DeleteSubjectKeync(subjectId);
190
191     // Assert
192     result.Should().BeTrue();
193 }
194
195 [Fact]
196 public async Task SearchSubjectKeync_DatabaseControlMaterializes()
197 {
198     // Arrange
199     var subjects = new List<Subject>
200     {
201         new Subject
202         {
203             Code = "MATHS",
204             Name = "Mathematics"
205         },
206     };
207
208     _response.Setup(r =>
209         r.SearchSubject(
210             It.IsAny<string>(),
211             It.IsAny<int>(),
212             It.IsAny<int>(),
213             It.IsAny<int>(),
214             It.IsAny<int>())
215         .ReturnsAsync(subjects));
216 }

```

```

70     Code = "",
71     Name = "",
72     Semester = 0,
73     Credits = 0,
74     WeeklyHours = 0
75 };
76
77 // Act
78 Func<Task> act = async () =>
79     await _service.CreateSubjectAsync(dto);
80
81 // Assert
82 await act.Should().ThrowAsync<Exception>();
83
84 [Fact]
85 [Obsolete]
86 public async Task GetSubjectByIdAsync_DeberiaRetornarMaterial()
87 {
88     // Arrange
89     var subjectId = Guid.NewGuid();
90
91     _repoMock.Setup(r => r.GetByIdAsync(subjectId))
92         .ReturnsAsync(new Subject
93         {
94             Id = subjectId,
95             Code = "MAT101",
96             Name = "Matematicas"
97         });
98
99     // Act
100     var result = await _service.GetSubjectByIdAsync(subjectId);
101
102     // Assert
103     result.Should().NotBeNull();
104     result.Code.Should().Be("MAT101");
105 }
106
107 [Fact]
108 [Obsolete]
109 public async Task GetSubjectByIdAsync_DeberiaControlarMaterialInexistente()
110 {
111     // Arrange
112     var subjectId = Guid.NewGuid();
113
114     _repoMock.Setup(r => r.GetByIdAsync(subjectId))
115         .ReturnsAsync(Subject.Null);
116
117     // Act
118     var result = await _service.GetSubjectByIdAsync(subjectId);
119
120     // Assert
121     result.Should().BeNull();
122 }
123
124 [Fact]
125 [Obsolete]
126 public async Task UpdateSubjectAsync_DeberiaActualizarMaterial()
127 {
128     // Arrange
129     var subjectId = Guid.NewGuid();
130
131     var subject = new Subject
132     {
133         Id = subjectId,
134         Code = "GLD",
135         Name = "Lógica"
136     };
137
138     var dto = new UpdateSubjectDto
139     {
140         Code = "HIS",
141         Name = "Historia",
142         Semester = 2,
143         Credits = 4,
144         WeeklyHours = 5,
145         Inactive = true
146     };
147
148     _repoMock.Setup(r => r.GetByIdAsync(subjectId))
149         .ReturnsAsync(subject);
150
151     // Act
152
153

```

```

207
208 // Act
209 var result = await _service.DeleteSubjectAsync(subjectId);
210
211 // Assert
212 result.Should().BeTrue();
213 }
214
215 [Fact]
216 GetSubject
217 public async Task SearchSubjectsAsync_DeberiaRetornarMateria()
218 {
219     // Arrange
220     var subjects = new List<Subject>
221     {
222         new Subject
223         {
224             Code = "MATE101",
225             Name = "Matemáticas"
226         },
227     };
228
229     _repository.Setup(r =>
230         r.SearchAsync(
231             It.IsAny<string>(),
232             It.IsAny<int>(),
233             It.IsAny<string>(),
234             It.IsAny<int>(),
235             It.IsAny<int>(),
236             It.IsAny<int>()
237         ).ReturnsAsync(subjects, 1));
238
239 // Act
240 var result = await _service.SearchSubjectsAsync(
241     "MAT",
242     null,
243     true,
244     1,
245     10);
246
247 // Assert
248 result.Items.Should().HaveCount(1);
249 }
250
251 [Fact]
252 CreateSubject
253 public async Task CreateSubjectAsync_DeberiaControlarErroresRepetitivos()
254 {
255     // Arrange
256     var dto = new CreateSubjectDto
257     {
258         Code = "MATE101",
259         Name = "Matemáticas",
260         Semester = 1,
261         Credits = 3,
262         WeeklyHours = 4
263     };
264
265     _repository.Setup(r => r.GetByCodeAsync(dto.Code))
266         .ThrowsAsync(new Exception("Database error"));
267
268 // Act
269 Func<Task> act = async () =>
270     await _service.CreateSubjectAsync(dto);
271
272 // Assert
273 await act.Should()
274     .ThrowsAsync<Exception>()
275     .WithMessage("Database error");
276
277 }
278

```

TeacherServiceTests

```
TeacherServiceTests.cs
ScheduleApp.Tests
1 using FluentAssertions;
2 using NLog;
3 using ScheduleApp.Application.DTOs;
4 using ScheduleApp.Application.Interfaces;
5 using ScheduleApp.Application.Services;
6 using ScheduleApp.Domain.Entities;
7
8 namespace ScheduleApp.Tests.Services
9 {
10     [TestFixture]
11     public class TeacherServiceTests
12     {
13         private readonly Mock<ITeacherRepository> _teacherRepositoryMock;
14         private readonly Mock<ISubjectRepository> _subjectRepositoryMock;
15         private readonly TeacherService _service;
16
17         [SetUp]
18         public void TeacherServiceTests()
19         {
20             _teacherRepositoryMock = new Mock<ITeacherRepository>();
21             _subjectRepositoryMock = new Mock<ISubjectRepository>();
22             _service = new TeacherService(
23                 _teacherRepositoryMock.Object,
24                 _subjectRepositoryMock.Object
25             );
26         }
27
28         [Fact]
29         [TestFixture]
30         public async Task CreateAsync_DeberiaCrearDocumentoCorrectamente()
31         {
32             // Arrange
33             var dto = new CreateTeacherDto
34             {
35                 FirstName = "Salome",
36                 LastName = "Carmena",
37                 Email = "salome@test.com",
38                 IdentityDocument = "123456",
39                 PhoneNumber = "1001234567",
40                 TeachingHours = 20,
41                 ContractType = "Tiempo Completo",
42                 Specialties = "Programación"
43             };
44             _teacherRepositoryMock
45                 .Setup(r => r.GetByIdAsync(dto.Email))
46                 .ReturnsAsync((Teacher?)null);
47             _teacherRepositoryMock
48                 .Setup(r => r.GetByIdAsync(dto.IdentityDocument))
49                 .ReturnsAsync((Teacher?)null);
50             _subjectRepositoryMock
51                 .Setup(r => r.SearchAsync(
52                     It.IsAny<string>(),
53                     true,
54                     1
55                 ))
56                 .ReturnsAsync(
57                     new List<Subject>
58                     {
59                         new Subject
60                         {
61                             Id = Guid.NewGuid(),
62                             Name = "Programación"
63                         },
64                     },
65                     1
66                 );
67             // Act
68             var result = await _service.CreateAsync(dto);
69             // Assert
70             result.Should().NotBeNull();
71             result.Email.Should().Be(dto.Email);
72             _teacherRepositoryMock.Verify(
73                 r => r.AddAsync(It.IsAny<Teacher>()),
74                 Times.Once
75             );
76         }
77     }
78 }
```

```
TeacherServiceTests.cs
ScheduleApp.Tests
81 }
82
83 [Fact]
84 public async Task CreateAsync_DeberiaLanzarError_SiYaExiste()
85 {
86     // Arrange
87     var dto = new CreateTeacherDto
88     {
89         Email = "duplicado@test.com",
90         IdentityDocument = "123"
91     };
92     _teacherRepositoryMock
93         .Setup(r => r.GetByIdAsync(dto.Email))
94         .ReturnsAsync(new Teacher());
95
96     // Act
97     Func<Task> action = async () => await _service.CreateAsync(dto);
98
99     // Assert
100     await action.Should()
101         .ThrowAsync<InvalidOperationException>()
102         .WithMessage("Ya existe un docente con esta identidad");
103 }
104
105 [Fact]
106 [TestFixture]
107 public async Task CreateAsync_DeberiaLanzarError_SiDocumentoExiste()
108 {
109     // Arrange
110     var dto = new CreateTeacherDto
111     {
112         Email = "nuevo@test.com",
113         IdentityDocument = "9999999"
114     };
115     _teacherRepositoryMock
116         .Setup(r => r.GetByIdAsync(dto.Email))
117         .ReturnsAsync((Teacher?)null);
118     _teacherRepositoryMock
119         .Setup(r => r.GetByIdAsync(dto.IdentityDocument))
120         .ReturnsAsync(new Teacher());
121
122     // Act
123     Func<Task> action = async () => await _service.CreateAsync(dto);
124
125     // Assert
126     await action.Should()
127         .ThrowAsync<InvalidOperationException>()
128         .WithMessage("Documento de identidad ya está registrado");
129 }
130
131 [Fact]
132 [TestFixture]
133 public async Task GetByIdAsync_DeberiaRetornarDocente()
134 {
135     // Arrange
136     var teacher = new Teacher
137     {
138         Id = Guid.NewGuid(),
139         FirstName = "Mateo",
140         LastName = "Quintero",
141         Email = "mateo@test.com",
142         IdentityDocument = "123",
143         PhoneNumber = "1000000000"
144     };
145     _teacherRepositoryMock
146         .Setup(r => r.GetByIdAsync(teacher.Id))
147         .ReturnsAsync(teacher);
148
149     // Act
150     var result = await _service.GetByIdAsync(teacher.Id);
151
152     // Assert
153     result.Should().NotBeNull();
154     result.Email.Should().Be(teacher.Email);
155 }
156
157 [Fact]
158 [TestFixture]
159 public async Task DeleteAsync_DeberiaDesactivarDocente()
160 {
161     // Arrange
162     var teacher = new Teacher
163     {
164         Id = Guid.NewGuid(),
165         IsActive = true
166     };
167     _teacherRepositoryMock
168         .Setup(r => r.GetByIdAsync(teacher.Id))
169         .ReturnsAsync(teacher);
170
171     // Act
172     var result = await _service.DeleteAsync(teacher.Id);
173
174     // Assert
175     result.Should().BeTrue();
176     teacher.IsActive.Should().BeFalse();
177     _teacherRepositoryMock.Verify(
178         r => r.UpdateAsync(It.IsAny<Teacher>()),
179         Times.Once
180     );
181 }
182
183 [Fact]
184 [TestFixture]
185 public async Task ChangeStatusAsync_DeberiaCambiarEstado()
186 {
187     // Arrange
188     var teacher = new Teacher
189     {
190         Id = Guid.NewGuid(),
191         IsActive = true
192     };
193     _teacherRepositoryMock
194         .Setup(r => r.GetByIdAsync(teacher.Id))
195         .ReturnsAsync(teacher);
196
197     // Act
198     var result = await _service.ChangeStatusAsync(teacher.Id, false);
199
200     // Assert
201     result.Should().BeTrue();
202     result.IsActive.Should().BeFalse();
203 }
204
205 [Fact]
206 [TestFixture]
207 public async Task SearchAsync_DeberiaRetornarDocentes()
208 {
209     // Arrange
210     var teachers = new List<Teacher>
211     {
212         new Teacher
213         {
214             Id = Guid.NewGuid(),
215             FirstName = "Salome",
216             LastName = "Carmena",
217             Email = "salome@test.com"
218         },
219     };
220     _teacherRepositoryMock
221         .Setup(r => r.SearchAsync(null, null))
222         .ReturnsAsync(teachers);
223
224     // Act
225     var result = await _service.SearchAsync(null, null, null);
226
227     // Assert
228     result.Should().NotBeNull();
229     result.Count().Should().Be(1);
230 }
231 }
```

```
TeacherServiceTests.cs
ScheduleApp.Tests
160 [TestFixture]
161 public async Task DeleteAsync_DeberiaDesactivarDocente()
162 {
163     // Arrange
164     var teacher = new Teacher
165     {
166         Id = Guid.NewGuid(),
167         IsActive = true
168     };
169     _teacherRepositoryMock
170         .Setup(r => r.GetByIdAsync(teacher.Id))
171         .ReturnsAsync(teacher);
172
173     // Act
174     var result = await _service.DeleteAsync(teacher.Id);
175
176     // Assert
177     result.Should().BeTrue();
178     teacher.IsActive.Should().BeFalse();
179     _teacherRepositoryMock.Verify(
180         r => r.UpdateAsync(It.IsAny<Teacher>()),
181         Times.Once
182     );
183 }
184
185 [Fact]
186 [TestFixture]
187 public async Task ChangeStatusAsync_DeberiaCambiarEstado()
188 {
189     // Arrange
190     var teacher = new Teacher
191     {
192         Id = Guid.NewGuid(),
193         IsActive = true
194     };
195     _teacherRepositoryMock
196         .Setup(r => r.GetByIdAsync(teacher.Id))
197         .ReturnsAsync(teacher);
198
199     // Act
200     var result = await _service.ChangeStatusAsync(teacher.Id, false);
201
202     // Assert
203     result.Should().BeTrue();
204     result.IsActive.Should().BeFalse();
205 }
206
207 [Fact]
208 [TestFixture]
209 public async Task SearchAsync_DeberiaRetornarDocentes()
210 {
211     // Arrange
212     var teachers = new List<Teacher>
213     {
214         new Teacher
215         {
216             Id = Guid.NewGuid(),
217             FirstName = "Salome",
218             LastName = "Carmena",
219             Email = "salome@test.com"
220         },
221     };
222     _teacherRepositoryMock
223         .Setup(r => r.SearchAsync(null, null))
224         .ReturnsAsync(teachers);
225
226     // Act
227     var result = await _service.SearchAsync(null, null, null);
228
229     // Assert
230     result.Should().NotBeNull();
231     result.Count().Should().Be(1);
232 }
233 }
```


UserServiceTests

```
UserServiceTests.cs
ScheduleApp.Tests
1 using FluentAssertions;
2 using NLog;
3 using ScheduleApp.Application.Interfaces;
4 using ScheduleApp.Application.Services;
5 using ScheduleApp.Domain.Entities;
6 using ScheduleApp.Tests.Services;
7
8 namespace ScheduleApp.Tests.Services;
9
10 // Autor: Juancho
11 // Version: 1.1
12 // Pruebas unitarias del UserService (M0-168)
13 // Cabeza los 6 casos de éxito y los 6 casos de error definidos en la M0
14 // Se usa NLog para simular el comportamiento de PasswordHasher, y PasswordGenerator
15 // Para hacer los asserts son legibles
16
17 public class UserServiceTests
18 {
19     private readonly IRepository<User> _userRepository;
20     private readonly PasswordHasher _passwordHasher;
21     private readonly UserService _userService;
22
23     private static readonly Guid AdministratorRoleId =
24         Guid.Parse("11111111-1111-1111-1111-111111111111");
25
26     private static readonly Guid CoordinatorRoleId =
27         Guid.Parse("22222222-2222-2222-2222-222222222222");
28
29     // Dependencias
30     public UserServiceTests()
31     {
32         _userRepository = new Mock<IRepository<User>>();
33         _passwordHasher = new Mock<IPasswordHasher>();
34         // Aquí siempre retorna un string, fijo para no depender de bcrypt en los tests
35         _passwordHasher.Setup(x => x.Hash(It.IsAny<string>()))
36             .Returns("hashed-password");
37         _userService = new UserService(_userRepository.Object, _passwordHasher.Object);
38     }
39
40     // CASOS DE ÉXITO (6)
41     // =====
42
43     // CASO 1: Crear usuario con datos válidos
44     [Fact]
45     public async Task CreateAsync_WithValidData_ShouldReturnUserDto()
46     {
47         var dto = BuildValidCreateDto();
48         _userRepository.Setup(x => x.GetByEmailAsync(It.IsAny<string>()))
49             .ReturnsAsync((User?)null);
50         _passwordHasher.Setup(x => x.AddAsync(It.IsAny<string>()))
51             .Returns(Task.CompletedTask);
52
53         var result = await _userService.CreateAsync(dto);
54         result.Should().NotBeNull();
55         result.Email.Should().Be("juan.perez@edu.co");
56         result.FullName.Should().Be("Juan Perez");
57         result.IsActive.Should().BeTrue();
58         _userRepository.Verify(x => x.AddAsync(It.IsAny<User>()), Times.Once);
59     }
60
61     // CASO 2: Consultar listado de usuarios
62     [Fact]
63     public async Task GetUsersAsync_ShouldReturnAllUsers()
64     {
65         var users = new List<User>
66         {
67             new() { Id = Guid.NewGuid(), FullName = "User 1", Email = "u1@test.com" },
68             new() { Id = Guid.NewGuid(), FullName = "User 2", Email = "u2@test.com" }
69         };
70         _userRepository.Setup(x => x.SearchUsersAsync(null, null, null))
71             .ReturnsAsync(users);
72
73         var result = await _userService.GetUsersAsync();
74         result.Should().NotBeNull();
75         result.Count().Should().Be(2);
76         result.Should().Contain(x => x.Email == "u1@test.com");
77     }
78 }
```

```
UserServiceTests.cs
ScheduleApp.Tests
79 // CASO 3: Obtener usuario por ID
80 [Fact]
81 public async Task GetUserByIdAsync_WhenUserExists_ShouldReturnUserDto()
82 {
83     var userId = Guid.NewGuid();
84     var role = new Role { Id = AdministratorRoleId, Name = "Administrador" };
85     var user = new User
86     {
87         Id = userId,
88         FullName = "Test User",
89         Email = "test@edu.co",
90         Username = "testuser",
91         IdentityDocument = "12345",
92         IsActive = true,
93         Role = role,
94         RoleId = role.Id,
95         CreatedAt = DateTime.UtcNow
96     };
97     _userRepository.Setup(x => x.GetByIdAsync(userId)).ReturnsAsync(user);
98
99     var result = await _userService.GetUserByIdAsync(userId);
100     result.Should().NotBeNull();
101     result.Id.Should().Be(userId);
102     result.Email.Should().Be("test@edu.co");
103     result.FullName.Should().Be("Administrador");
104 }
105
106 // CASO 4: Actualizar usuario existente
107 [Fact]
108 public async Task UpdateUserAsync_WhenUserExists_ShouldUpdateAndReturnDto()
109 {
110     var userId = Guid.NewGuid();
111     var existingUser = new User
112     {
113         Id = userId,
114         FullName = "Old Name",
115         Email = "old@test.com",
116         Username = "olduser",
117         IdentityDocument = "111",
118         IsActive = true,
119         RoleId = CoordinatorRoleId
120     };
121     var updateDto = new UpdateUserDto
122     {
123         FullName = "New Name",
124         Email = "new@test.com",
125         Username = "newuser",
126         IdentityDocument = "222",
127         Role = "Coordinator",
128         IsActive = true
129     };
130     _userRepository.Setup(x => x.GetByIdAsync(userId)).ReturnsAsync(existingUser);
131     _userRepository.Setup(x => x.GetByEmailAsync(It.IsAny<string>()))
132         .ReturnsAsync((User?)null);
133     _passwordHasher.Setup(x => x.UpdateAsync(It.IsAny<User>()))
134         .Returns(Task.CompletedTask);
135
136     var result = await _userService.UpdateUserAsync(userId, updateDto);
137     result.Should().NotBeNull();
138     result.FullName.Should().Be("New Name");
139     result.Email.Should().Be("new@test.com");
140     _userRepository.Verify(x => x.UpdateAsync(It.IsAny<User>()), Times.Once);
141 }
142
143 // CASO 5: Eliminar / desactivar usuario
144 [Fact]
145 public async Task DeleteUserAsync_WhenUserActive_ShouldDeactivateAndReturnTrue()
146 {
147     var userId = Guid.NewGuid();
148     var user = new User
149     {
150         Id = userId,
151         FullName = "Test",
152         Email = "test@test.com",
153         IsActive = true
154     };
155 }
```

```
UserServiceTests.cs
ScheduleApp.Tests
157 _userRepository.Setup(x => x.GetByIdAsync(userId)).ReturnsAsync(user);
158 _passwordHasher.Setup(x => x.DeleteAsync(It.IsAny<string>()))
159     .Returns(Task.CompletedTask);
160
161 var result = await _userService.DeleteUserAsync(userId);
162 result.Should().BeTrue();
163 user.IsActive.Should().BeFalse();
164 _userRepository.Verify(x => x.UpdateAsync(It.IsAny<User>() => !x.IsActive), Times.Once);
165
166 // CASO 6: Asignar rol a un usuario
167 [Fact]
168 public async Task CreateAsync_WithAdminRole_ShouldAssignAdminRole()
169 {
170     var dto = BuildValidCreateDto();
171     dto.Role = "Administrator";
172     _userRepository.Setup(x => x.GetByEmailAsync(It.IsAny<string>()))
173         .ReturnsAsync((User?)null);
174     _passwordHasher.Setup(x => x.AddAsync(It.IsAny<string>()))
175         .Returns(Task.CompletedTask);
176     _userRepository.Setup(x => x.AddAsync(It.IsAny<User>()))
177         .ReturnsAsync((User?)null);
178     _passwordHasher.Setup(x => x.CaptureUser(a => a.RoleId == a))
179         .Returns(Task.CompletedTask);
180
181     await _userService.CreateAsync(dto);
182     _userRepository.Setup(x => x.GetByIdAsync(userId)).ReturnsAsync(user);
183     user.RoleId.Should().Be(AdministratorRoleId);
184
185     // CASO 7: Búsqueda + filtros + paginación
186     [Fact]
187     public async Task GetPaginatedUsers_WithFilters_ShouldReturnPageResultWithMetadata()
188     {
189         var users = new List<User>
190         {
191             new() { Id = Guid.NewGuid(), FullName = "Admin User", Email = "admin@test.com", IsActive = true },
192             new() { Id = Guid.NewGuid(), FullName = "Admin", Email = "Admin", IsActive = true, 1, 10 }
193         };
194         _userRepository.Setup(x => x.GetPaginatedUsers("Admin", "Administrador", true, 1, 10))
195             .ReturnsAsync((users, 1));
196
197         var result = await _userService.GetPaginatedUsersAsync("Admin", "Administrador", true, 1, 10);
198         result.Should().NotBeNull();
199         result.TotalCount.Should().Be(1);
200         result.Page.Should().Be(1);
201         result.PageSize.Should().Be(10);
202         result.TotalCount.Should().Be(1);
203         result.TotalPages.Should().Be(1);
204     }
205
206 // CASO 8: Datos retornados corresponden a la información registrada
207 [Fact]
208 public async Task GetUserByIdAsync_ShouldReturnUserWithCorrectFields()
209 {
210     var registerEmail = "register@edu.co";
211     var registerId = Guid.NewGuid();
212     var users = new List<User>
213     {
214         new()
215         {
216             Id = registerId,
217             FullName = "Registered User",
218             Email = registerEmail,
219             Username = "register",
220             IdentityDocument = "SSP",
221             IsActive = true,
222             CreatedAt = new DateTime(2024, 5, 1)
223         }
224     };
225     _userRepository.Setup(x => x.SearchUsersAsync(null, null, null))
226         .ReturnsAsync(users);
227
228     var result = await _userService.GetUserByIdAsync(registerId);
229     var first = result.First();
230     first.Id.Should().Be(registerId);
231     first.Email.Should().Be(registerEmail);
232     first.FullName.Should().Be(registerEmail);
233 }
```

```
UserServiceTests.cs
ScheduleApp.Tests
234 first.Id.Should().Be(registerId);
235 first.Email.Should().Be(registerEmail);
236 first.FullName.Should().Be(registerEmail);
237 first.Username.Should().Be("register");
238
239 // CASOS DE ERROR (6)
240 // =====
241
242 // Error 1: No se permite crear con correo duplicado
243 [Fact]
244 public async Task CreateAsync_WithDuplicateEmail_ShouldThrowInvalidOperationException()
245 {
246     var dto = BuildValidCreateDto();
247     var existing = new User { Id = Guid.NewGuid(), Email = dto.Email.ToLower().Trim() };
248     _userRepository.Setup(x => x.GetByEmailAsync(It.IsAny<string>()))
249         .ReturnsAsync(existing);
250
251     Func<Task> act = async () => await _userService.CreateAsync(dto);
252
253     await act.Should().ThrowAsync<InvalidOperationException>()
254         .WithMessage("Ya existe un usuario");
255     _passwordHasher.Verify(x => x.AddAsync(It.IsAny<User>()), Times.Never);
256 }
257
258 // Error 2: Si el repositorio lanza (ej: NOL DATA por campos obligatorios vacíos),
259 // el servicio lo propaga al controller
260 [Fact]
261 public async Task CreateAsync_WhenRepositoryThrows_ShouldPropagateError()
262 {
263     var dto = BuildValidCreateDto();
264     _userRepository.Setup(x => x.GetByEmailAsync(It.IsAny<string>()))
265         .ReturnsAsync((User?)null);
266     _passwordHasher.Setup(x => x.AddAsync(It.IsAny<User>()))
267         .ThrowsAsync(new InvalidOperationException("DB constraint violation"));
268
269     Func<Task> act = async () => await _userService.CreateAsync(dto);
270
271     await act.Should().ThrowAsync<InvalidOperationException>();
272 }
273
274 // Error 3: No se permite actualizar a un email ya usado por otro usuario
275 [Fact]
276 public async Task UpdateUserAsync_WithDuplicateEmail_ShouldThrowInvalidOperationException()
277 {
278     var userId = Guid.NewGuid();
279     var otherUserId = Guid.NewGuid();
280     var existingUser = new User
281     {
282         Id = userId,
283         FullName = "User A",
284         Email = "a@test.com",
285         IsActive = true
286     };
287     var anotherUserWithMail = new User
288     {
289         Id = otherUserId,
290         Email = "duplicate@test.com"
291     };
292     var updateDto = new UpdateUserDto
293     {
294         FullName = "User A",
295         Email = "duplicate@test.com",
296         Username = "usera",
297         IdentityDocument = "123",
298         Role = "Coordinator",
299         IsActive = true
300     };
301     _userRepository.Setup(x => x.GetByIdAsync(userId)).ReturnsAsync(existingUser);
302     _userRepository.Setup(x => x.GetByEmailAsync(It.IsAny<string>()))
303         .ReturnsAsync(anotherUserWithMail);
304
305     Func<Task> act = async () => await _userService.UpdateUserAsync(userId, updateDto);
306
307     await act.Should().ThrowAsync<InvalidOperationException>()
308         .WithMessage("Ya existe otro usuario");
309 }
```



```
UserServiceTests.cs
ScheduleApp.Tests

110 }
111
112 // Error 4: Consulta de usuario inexistente devuelve null (no lanza).
113 [Fact]
114 [Obsolete]
115 public async Task GetByIdAsync_WhenUserNotExists_ShouldReturnNull()
116 {
117     var userId = Guid.NewGuid();
118     _userRepository.Setup(r => r.GetByIdAsync(userId)).ReturnsAsync((User?)null);
119     var result = await _userService.GetByIdAsync(userId);
120     result.Should().BeNull();
121 }
122
123 // Error 5: Update de usuario inexistente devuelve null y no llama al repo.
124 [Fact]
125 [Obsolete]
126 public async Task UpdateUserAsync_WhenUserNotExists_ShouldReturnNull()
127 {
128     var userId = Guid.NewGuid();
129     var updateDto = new UpdateUserDto
130     {
131         FullName = "JohnDoe",
132         Email = "johndoe.com",
133         Username = "jd",
134         IdentityDocument = "n",
135         Role = "Coordinator",
136         IsActive = true
137     };
138     _userRepository.Setup(r => r.GetByIdAsync(userId)).ReturnsAsync((User?)null);
139     var result = await _userService.UpdateUserAsync(userId, updateDto);
140     result.Should().BeNull();
141     _userRepository.Verify(r => r.UpdateAsync(It.IsAny<User>()), Times.Never);
142 }
143
144 // Error 6: Delete de usuario inexistente devuelve false y no llama al repo.
145 [Fact]
146 [Obsolete]
147 public async Task DeleteUserAsync_WhenUserNotExists_ShouldReturnFalse()
148 {
149     var userId = Guid.NewGuid();
150     _userRepository.Setup(r => r.GetByIdAsync(userId)).ReturnsAsync((User?)null);
151     var result = await _userService.DeleteUserAsync(userId);
152     result.Should().BeFalse();
153     _userRepository.Verify(r => r.UpdateAsync(It.IsAny<User>()), Times.Never);
154 }
155
156 // HELPER
157 //
158 [Obsolete]
159 private static CreateUserDto BuildValidCreateDto()
160 {
161     return new CreateUserDto
162     {
163         FullName = "Juan Perez",
164         Email = "juan.perez@juan.edu.es",
165         Username = "jperez",
166         IdentityDocument = "1234567890",
167         Password = "Password123",
168         ConfirmPassword = "Password123",
169         Role = "Coordinator",
170         Status = "Active"
171     };
172 }
```

Captura de ejecución de dotnet test y resultado final

```
Archivo Editor Ver Git Proyecto Compilar Depurar Prueba Herramientas Extensiones Ventana Ayuda Q Buscar ScheduleApp
ScheduleApp.WebApi https://...
AuthServiceTests
ScheduleApp.Tests
ScheduleApp.Application
ScheduleApp.Application.Services.AuthService
LoginAsync(LoginRequestDto request)
Explorador de soluciones
Buscar en Explorador de soluciones (Ctrl+...)
> C# ISubjectRepository.cs
> C# IAuthService.cs
> C# IApiResponseRepository.cs
Developer PowerShell
+ Developer PowerShell
PS C:\Users\juane\source\repos\ScheduleApp-Backend> dotnet test
Restauración completada (2,3s)
ScheduleApp.Domain net10.0 realizado correctamente (3,9s) -> src\ScheduleApp.Domain\bin\Debug\net10.0\ScheduleApp.Domain.dll
ScheduleApp.Application net10.0 realizado correctamente (1,4s) -> src\ScheduleApp.Application\bin\Debug\net10.0\ScheduleApp.Application.dll
ScheduleApp.Infrastructure net10.0 correcto con 3 advertencias (2,5s) -> src\ScheduleApp.Infrastructure\bin\Debug\net10.0\ScheduleApp.Infrastructure.dll
C:\Users\juane\source\repos\ScheduleApp-Backend\src\ScheduleApp.Infrastructure\Repositorios\ScheduleRepository.cs(219,35): warning CS8602: Desreferencia de una referencia posiblemente NULL.
C:\Users\juane\source\repos\ScheduleApp-Backend\src\ScheduleApp.Infrastructure\Repositorios\ScheduleRepository.cs(225,39): warning CS8602: Desreferencia de una referencia posiblemente NULL.
C:\Users\juane\source\repos\ScheduleApp-Backend\src\ScheduleApp.Infrastructure\Repositorios\ScheduleRepository.cs(228,37): warning CS8602: Desreferencia de una referencia posiblemente NULL.
ScheduleApp.Tests net10.0 realizado correctamente (2,1s) -> ScheduleApp.Tests\bin\Debug\net10.0\ScheduleApp.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4 (64-bit .NET 10.0.0)
[xUnit.net 00:00:00.21] Discovering: ScheduleApp.Tests
[xUnit.net 00:00:00.31] Discovered: ScheduleApp.Tests
[xUnit.net 00:00:00.36] Starting: ScheduleApp.Tests
Warning:
The component "Fluent Assertions" is governed by the rules defined in the Xceed License Agreement and
the Xceed Fluent Assertions Community License. You may use Fluent Assertions free of charge for
non-commercial use only. An active subscription is required to use Fluent Assertions for commercial use.
Please contact Xceed Sales mullto@xceed.com to acquire a subscription at a very low cost.
A paid commercial license supports the development and continued increasing support of
Fluent Assertions users under both commercial and community licenses. Help us
keep Fluent Assertions at the forefront of unit testing.
For more information, visit https://xceed.com/products/unit-testing/Fluent-assertions/
[xUnit.net 00:00:00.71] Finished: ScheduleApp.Tests
ScheduleApp.Tests net10.0 realizado correctamente prueba (2,4s)
Resumen de pruebas: total: 65; con errores: 0; correcto: 65; omitido: 0; duración: 2,4 s
Compilación correcto con 3 advertencias en 16,5s
PS C:\Users\juane\source\repos\ScheduleApp-Backend>
```